

Classe DALPro

A classe DALPro implementa uma camada de acesso a dados (Data Access Layer) simplificada para aplicações .NET que utilizam SQL Server.

- Fornece métodos utilitários para: execução de comandos SQL (INSERT, UPDATE, DELETE), obtenção de valores escalares (refere-se a um único dado individual retornado por uma consulta, em vez de uma tabela inteira ou um conjunto de registros), leitura de dados com mapeamento automático para objetos, execução de **Stored Procedures**, manipulação de dados via DataTable/DataAdapter, gestão de **transações**.

- Vantagens da classe: reduz muito o código repetido de ADO.NET, suporte nativo a transações, mapeamento automático de objetos, suporte a parâmetros SQL, suporte a Stored Procedures, cache de reflection para melhor performance.

CampoConnectionString

```
public static string ConnectionString;
```

Descrição: Contém a string de ligação à base de dados utilizada por todos os métodos da classe.

Exemplo : DALPro.ConnectionString =

```
Server=localhost;Database=Northwind;Trusted_Connection=True;TrustServerCertificate=True;;
```

Método GetConnection

```
public static SqlConnection GetConnection()
```

Descrição : Cria uma nova instância de SqlConnection usando a ConnectionString.

Retorna: SqlConnection não aberta.

Exemplo: using var cn = DALPro.GetConnection();
cn.Open();

Método CreateCommand

```
private static SqlCommand CreateCommand(  
    string sql, SqlTransaction trans = null,  
    Dictionary<string, object> parameters = null)
```

Descrição: Cria e configura um objeto SqlCommand.

Parâmetros:

Parâmetro	Descrição
sql	Query SQL ou comando a executar
trans	Transação opcional
parameters	Lista de parâmetros SQL

Comportamento:

Se existir transação: cmd = new SqlCommand(sql, trans.Connection, trans)

Caso contrário: cria nova ligação, abre a ligação, associa ao comando

Utilização: Método interno usado por: Execute, ExecuteScalar, Query<T>.

Método Execute

```
public static int Execute(  
    string sql, Dictionary<string, object> parameters = null,  
    SqlTransaction trans = null)
```

Descrição : Executa comandos SQL que não retornam resultados.

Uso típico: INSERT, UPDATE, DELETE, DDL.

Retorna: Número de linhas afetadas.

Exemplo: DALPro.Execute("UPDATE Products SET UnitPrice = UnitPrice + 1 WHERE ProductID=@id", new Dictionary<string, object> { { "@id", 1 } });

Método ExecuteScalar

```
public static object ExecuteScalar(  
    string sql, Dictionary<string, object> parameters = null,  
    SqlTransaction trans = null)
```

Descrição : Executa uma query que retorna um único valor.

Uso típico: COUNT, SUM, AVG, SCOPE_IDENTITY().

Retorna: Primeira coluna da primeira linha do resultado.

Exemplo: int total = Convert.ToInt32(DALPro.ExecuteScalar("SELECT COUNT(*) FROM Products"));

Método Query<T>

```
public static List<T> Query<T>(  
    string sql, Dictionary<string, object> parameters = null,  
    SqlTransaction trans = null)
```

```

string sql,
Dictionary<string, object> parameters = null,
SqlTransaction trans = null) where T : new()

```

Descrição

Executa uma query SQL e converte automaticamente os resultados numa lista de objetos.

Funcionamento: Executa o SQL, Lê cada linha do SqlDataReader, Usa reflection para mapear colunas → propriedades, Cria um objeto T por cada linha.

Requisitos: O nome das propriedades deve corresponder ao nome das colunas.

Retorna: Lista de objetos List<T>.

Exemplo: Classe modelo:

```

public class Product
{
    public int ProductID { get; set; }
    public string ProductName { get; set; }
}

```

Consulta:

```

var produtos = DALPro.Query<Product>(
"SELECT ProductID, ProductName FROM Products");

```

Método DataTableForUpdate

```

public static DataTable DataTableForUpdate(
    string sql,
    ref SqlDataAdapter da,
    SqlTransaction trans = null)

```

Descrição: Obtém dados numa DataTable permitindo edição e atualização automática na base de dados.

Funcionamento: cria SqlDataAdapter, associa SqlCommandBuilder. carrega dados para DataTable, devolve a tabela.

Atualização: Depois de alterar dados: da.Update(dt);

Exemplo SqlDataAdapter da = null;

```

    DataTable dt = DALPro.DataTableForUpdate(
        "SELECT ProductID, ProductName FROM Products",
        ref da);
    dt.Rows[0]["ProductName"] = "Novo Produto";
    da.Update(dt);

```

Método ExecuteSP

```

public static DataTable ExecuteSP(
    string spName,
    Dictionary<string, object> parameters = null,
    SqlTransaction trans = null)

```

Descrição: Executa uma Stored Procedure.

Retorna: Resultado numa DataTable.

Exemplo: var prm = new Dictionary<string, object>

```

{
    { "@CustomerID", "ALFKI" }
};

```

```

    DataTable dt = DALPro.ExecuteSP("CustOrdersOrders", prm);

```

Método BeginTransaction

```

public static SqlTransaction BeginTransaction()

```

Descrição: Inicia uma nova transação na base de dados.

Retorna: Objeto SqlTransaction.

Exemplo: var trans = DALPro.BeginTransaction();

Método Commit

```

public static void Commit(SqlTransaction trans)

```

Descrição: Confirma todas as operações executadas dentro da transação.

Comportamento: executa Commit, fecha a ligação associada

Exemplo: DALPro.Commit(trans);

Método Rollback

public static void Rollback(SqlTransaction trans)

Descrição: Cancela todas as operações realizadas na transação.

Comportamento: executa Rollback, fecha a ligação

Exemplo: DALPro.Rollback(trans);

```
using System.Data;
using System.Reflection;
using Microsoft.Data.SqlClient;

namespace DalProLib
{
    0 referências
    public class DalPro
    {
        public static string ConnectionString;
        private static readonly Dictionary<Type, PropertyInfo[]> _cacheProps =
            new Dictionary<Type, PropertyInfo[]>();
        6 referências
        public static SqlConnection GetConnection()
        {
            return new SqlConnection(ConnectionString);
        }
        0 referências
        public static bool TryConnect()
        {
            try
            {
                using var cn = GetConnection();
                cn.Open();
                return true;
            }
            catch
            {
                return false;
            }
        }
    }
}
```

```
public static int Execute(
    string sql,
    Dictionary<string, object> parameters = null,
    SqlTransaction trans = null)
{
    using SqlCommand cmd = CreateCommand(sql, trans, parameters);
    int result = cmd.ExecuteNonQuery();
    if (trans == null)
        cmd.Connection.Close();
    return result;
}
```

```
public static DataTable ExecuteSP(
    string spName,
    Dictionary<string, object> parameters = null,
    SqlTransaction trans = null)
{
    SqlConnection cn;
    if (trans != null)
        cn = trans.Connection;
    else
    {
        cn = GetConnection();
        cn.Open();
    }
    SqlCommand cmd = new SqlCommand(spName, cn);
    cmd.CommandType = CommandType.StoredProcedure;
    if (trans != null)
        cmd.Transaction = trans;
    if (parameters != null)
    {
        foreach (var p in parameters)
            cmd.Parameters.AddWithValue(p.Key, p.Value ?? DBNull.Value);
    }
    SqlDataAdapter da = new SqlDataAdapter(cmd);
    DataTable dt = new DataTable();
    da.Fill(dt);
    if (trans == null)
        cn.Close();
    return dt;
}
```

```
public static object ExecuteScalar(
    string sql,
    Dictionary<string, object> parameters = null,
    SqlTransaction trans = null)
{
    using SqlCommand cmd = CreateCommand(sql, trans, parameters);
    object result = cmd.ExecuteScalar();
    if (trans == null)
        cmd.Connection.Close();
    return result;
}
```

```
private static SqlCommand CreateCommand(
    string sql,
    SqlTransaction trans = null,
    Dictionary<string, object> parameters = null)
{
    SqlCommand cmd;
    if (trans != null)
        cmd = new SqlCommand(sql, trans.Connection, trans);
    else
    {
        SqlConnection cn = GetConnection();
        cn.Open();
        cmd = new SqlCommand(sql, cn);
    }
    if (parameters != null)
    {
        foreach (var p in parameters)
            cmd.Parameters.AddWithValue(p.Key, p.Value ?? DBNull.Value);
    }
    return cmd;
}
```

```
public static DataTable DataTableForUpdate(
    string sql,
    ref SqlDataAdapter da,
    SqlTransaction trans = null)
{
    SqlConnection cn;

    if (trans != null)
        cn = trans.Connection;
    else
    {
        cn = GetConnection();
        cn.Open();
    }
    da = new SqlDataAdapter(sql, cn);
    if (trans != null)
        da.SelectCommand.Transaction = trans;
    da.MissingSchemaAction = MissingSchemaAction.AddWithKey;
    SqlCommandBuilder cb = new SqlCommandBuilder(da);
    cb.QuotePrefix = "[";
    cb.QuoteSuffix = "]";
    DataTable dt = new DataTable();
    da.Fill(dt);
    if (trans == null)
        cn.Close();
    return dt;
}
```

```
public static List<T> Query<T>(
    string sql,
    Dictionary<string, object> parameters = null,
    SqlTransaction trans = null) where T : new()
{
    List<T> list = new();
    using SqlCommand cmd = CreateCommand(sql, trans, parameters);
    using SqlDataReader dr = cmd.ExecuteReader();
    PropertyInfo[] props;
    if (!_cacheProps.TryGetValue(typeof(T), out props))
    {
        props = typeof(T).GetProperties();
        _cacheProps[typeof(T)] = props;
    }
    while (dr.Read())
    {
        T obj = new T();
        foreach (var prop in props)
        {
            try
            {
                int idx = dr.GetOrdinal(prop.Name);

                if (!dr.IsDBNull(idx))
                    prop.SetValue(obj, dr[idx]);
            }
            catch { }
        }
        list.Add(obj);
    }
    if (trans == null)
        cmd.Connection.Close();
    return list;
}
```

```

public static object ExecuteScalarSP(
    string spName,
    Dictionary<string, object> parameters = null,
    SqlTransaction trans = null)
{
    SqlConnection cn;
    if (trans != null)
        cn = trans.Connection;
    else
    {
        cn = GetConnection();
        cn.Open();
    }
    SqlCommand cmd = new SqlCommand(spName, cn);
    cmd.CommandType = CommandType.StoredProcedure;
    if (trans != null)
        cmd.Transaction = trans;
    if (parameters != null)
    {
        foreach (var p in parameters)
            cmd.Parameters.AddWithValue(p.Key, p.Value ?? DBNull.Value);
    }
    object result = cmd.ExecuteScalar();
    if (trans == null)
        cn.Close();
    return result;
}

```

```

public static SqlTransaction BeginTransaction()
{
    SqlConnection cn = GetConnection();
    cn.Open();
    return cn.BeginTransaction();
}
0 referências
public static void Commit(SqlTransaction trans)
{
    SqlConnection cn = trans.Connection;
    trans.Commit();
    if (cn.State == ConnectionState.Open)
        cn.Close();
}
0 referências
public static void Rollback(SqlTransaction trans)
{
    SqlConnection cn = trans.Connection;
    trans.Rollback();
    if (cn.State == ConnectionState.Open)
        cn.Close();
}

```